Advanced Binary Tree Node Counting and Verification Using C++

Problem Statement

Write an advanced C++ program to create a binary tree and perform detailed node counting and classification using both recursive and iterative traversal techniques. The program should calculate the total number of nodes, identify and display the values of all leaf nodes, count internal nodes, count nodes having exactly one child, and count nodes having exactly two children.

A leaf node is a node with no children, an internal node is a node with at least one child, a one-child node has exactly one child, and a two-child node has both left and right children.

The program should define a binary tree node structure containing a data field and pointers to the left and right children. After creating the binary tree, it should traverse the tree recursively to classify each node and update the corresponding counters. During traversal, the program should also display the values of all leaf nodes rather than displaying only their total count.

Additionally, the program should implement an iterative traversal method using a queue to calculate the total number of nodes independently. It should then compare the total number of nodes obtained through recursive and iterative methods. If both counts are equal, the program should display a message confirming that the results match. Otherwise, it should indicate that the counts differ.

Finally, the program should display the total number of nodes, leaf nodes, internal nodes, nodes having exactly one child, and nodes having exactly two children, along with the values of the leaf nodes.

Objectives

- To understand the structure and working of a binary tree.
- To create and manage a binary tree using C++.
- To count the total number of nodes using recursion.
- To count the total number of nodes using an iterative method with a queue.
- To identify and display the values of all leaf nodes.
- To count internal nodes having at least one child.
- To count nodes having exactly one child.
- To count nodes having exactly two children.
- To compare the results of recursive and iterative node counting.
- To verify the correctness of the total node count.
- To improve understanding of recursive and iterative tree traversal techniques.